Single Column Model (SCM) User Guide

This documentation is designed for the majority of SCM users. Since the primary function of the SCM is to serve as a simple platform for the development and analysis of physical parameterization schemes, this user guide focuses on descriptions of how to run the SCM in a variety of configurations. For developers of the SCM itself, and for SCM package managers, a separate set of Technical Documetation is available that provides detailed descriptions of the SCM code-base and its management.

Sommaire

QuickStart Guide

A set of benchmark data and configurations are provided to allow users to get up-and-running quickly with the SCM. For additional help for specific problems, consult the SCM Knowledge Base. By following the steps listed here, you will run the SCM for the GABLS-3 benchmark case. This 24-h simulation is the basis for an intercomparison project for column-based models and produces a set of well-documented results. The end result of these commands will be a plot of the near-surface temperature (TJ) time series for the full period of the study.

For GEM users, these commands will seem very familiar. In fact, the workflow involved with SCM integration has been designed to mirror that of the GEM model itself. This allows for a rapid learning curve for users already familiar with the tools required to run GEM. The first step is to acquire the SCM package.

Stable Version	Dev Version
`. r.load.dot SCM/x/2.1.0-rc9`	`. r.load.dot eccc/mrd/rpn/MIG/ENV/modeltools/scm_2.3.0-a13`

You may need to export this if you get an error message loading the ssm

```
export SSM_SHORTCUT_PATH=/fs/ssm/eccc/mrd/rpn/MIG:${SSM_SHORTCUT_PATH}
```

IMPORTANT: maestro needs to be loaded before running the SCM.

Thereafter, the commands to create a working directory are essentially identical to those used for GEM. For version 2.0-a4 and onwards, use the simplified configuration instructions to create and run an experiment called MY_EXP (replace this with your own more descriptive experiment name).

```
scmdev MY_EXP --cd -v
make cmake
make -j6 work
runscm
tsplot --var=TJ results/gabls-3
```

For versions older versions beyond 1.5-a2, the development environment employed more extensive configuration instructions.

```
ouv_exp_scm
linkit
scmcase -get gabls-3
runscm
tsplot --var=TJ results/gabls-3
```

Analogous instructions for older versions are shown as part of the more detailed SCM instructions.

General Description of the SCM

The SCM uses a simple two time-level forward time-stepping scheme to advance a set of equations for temperature, momentum and tracers over time. A set of dynamical forcings (i.e. horizontal advection terms) must be prescribed since they cannot be evaluated in the context of a single column. Additionally, there is no predictive equation for vertical motion or column mass (surface pressure), meaning that these fields must also be prescribed. In addition to the dynamical forcings applied each time-step, the tendencies from the physics package are applied. This allows for feed-back in the evolution of the profile since changes in the physics will lead to changes in the evolution of the model state.

The basic equations of the SCM are,

$$\frac{\partial T}{\partial t} = -V \cdot \nabla T - \omega \left(\frac{\partial T}{\partial p} - \frac{R}{c_p} \frac{T}{p} \right) + \frac{1}{\tau} (T_b - T) + F_{T_{phy}}$$

$$\frac{\partial u}{\partial t} = -V \cdot \nabla u - \omega \frac{\partial u}{\partial p} + f (v - v_g) + \frac{1}{\tau} (u_b - u) + F_{u_{phy}}$$

$$\frac{\partial v}{\partial t} = -V \cdot \nabla v - \omega \frac{\partial v}{\partial p} - f (u - u_g) + \frac{1}{\tau} (v_b - v) + F_{v_{phy}}$$

$$\frac{\partial X}{\partial t} = -V \cdot \nabla X - \omega \frac{\partial X}{\partial p} + \frac{1}{\tau} (X_b - X) + F_{X_{phy}}$$

Here, T is temperature, u and v are the westerly and southerly wind components, respectively, p is the pressure, ω is the pressure-coordinate vertical motion, R is the gas constant for dry air, c_p is the specific heat at constant pressure, and X represents any tracer. Wind components with a $\cdot_g$ subscript represent corresponding geostrophic wind (default 0ms^{-1}), while all variables with a $\cdot_b$ subscript refer to the "background" state provided by the driving model. The factor τ therefore represents a relaxation timescale to the evolving reference (driving model) profile (by default, $\frac{1}{\tau}$= 0s^{-1} so that no relaxation is performed). The F terms in the SCM equations correspond to the physical tendencies obtained by running the RPN physics package on the SCM profile. It is these terms that that make the SCM most useful since they allow the model state to evolve depending on the tendencies returned by the chosen physcial parameterizations.

In the discussion of SCM inputs below, a distinction is made between "Dynamics Inputs" and "Physics Inputs". In the context of the former, a set of forcing terms are discussed. These forcing terms are all of the terms on the right-hand side of the equations above, except for vertical advections and the physical tendencies (the latter appear at the end of the equations and denoted with F). The SCM treats each forcing separately as written in the equations above. This is particularly relevant for the advection terms: horizontal advections are provided as inputs to the SCM, while vertical advection is computed within the SCM dynamics.

The latest stable release of the SCM package is vx/2.1.0-rc9, to which this document applies. Since the SCM makes use of the RPN Physics API, any changes to the interface of the RPN Physics package necessitate an new version of the SCM. This means that any SCM release only works with a limited

number (possibly as few as one) of physics versions. A description of physics interoperability is provided later in this guide. In instances where major new features have been made available, the SCM version in which these features were implemented is noted.

Getting Started

There are a few steps involved with environment setup and work directory creation for the SCM. By design, these steps are very similar to those needed to run GEM. First, the SCM should be acquired in your environment by loading the SCM SSM package.

Stable Version	Dev Version
`. r.load.dot SCM/x/2.1.0-rc9`	`. r.load.dot eccc/mrd/rpn/MIG/ENV/modeltools/scm_2.3.0-a13`

Next, a working (or "experiment") directory should be created. Large files (object files, executables, data files) are stored separately in the location specified by the `$storage_model` environment variable, while `$scm_results` is used to control the default destination for output from SCM integrations. The working directory can therefore be located on a file system with limited space, preferably on a backed-up (snapshot) file system. By default, the working directory for experiment MY_EXP (a bad name used only as an example) is created in a path that contains the current SCM version information (shown as VERSION in the example) at `${HOME}/home/scm/VERSION/MY_EXP`. This path can be changed by providing a `-exp` argument to the command below (run `scmdev -h` for a full list of options). Adopting a uniform working directory structure makes it easier to compare configurations and results between users.

```
scmdev MY_EXP --cd -v
```

This command is common to the establishment of all working directories for the SCM. In addition to the creation of the working directory and build paths, a benchmark configuration is loaded into the working directory. By default, the GABLS-3 case is acquired; however, this can be changed using the `-case` option to `scmdev`, or at a later time using `scmcase`. As described in more detail in the section on running the SCM, two files are used by the SCM: `configexp.cfg` and `scm_settings.nml`. The former contains argument values for the `runscm` script, while the latter contains the `&scm_cfgs` and `&physics` namelists treated by the SCM. Input data can also be obtained for the benchmark cases as shown in the

QuickStart; however, it can also be generated as described in detail om the input data section of this guide. Once the SCM integration is complete, results reside in the `results` subdirectory and can be viewed and manipulated as described in the output processing section, below.

User Environment

Minimal environment configuration is required for the SCM, with more detailed user configurations residing in the `~/.scmrc` file. A pair of environment variables control the setup of the development environment.

- **scm_results**: (Optional) The base path of a directory in which SCM results will appear by default. If provided, this will be the base path of a directory that is the target of the `results/` link in the experiment directory.
- **storage_model**: The base path of a directory in which model components will be built, and in which the SCM is run. This directory should be visible from the local host, have rapid access and be prepared to handle a sufficient volume of data. This variable therefore usually points to a local disk. Because collisions between storage spaces for different experiments that share the same name is avoided by appending a hash based on the working directory path, the contents of the `storage_model` directory do not map directly to experiment names. For this reason, `devrm` is provided to help with storage cleanup.

Instructions for Older Versions

Simplified instructions for running the most recent version of the SCM are shown in the QuickStart. Instructions for older versions are shown here for reference. For version between 1.4.0-rc1 and 1.4.0, these RDE-based instructions are valid.

```
ouv_exp_scm
linkit
make dep
cp ${SCM_MODEL_DFILES}/run_configs/gabls-3/* .
runscm
tsplot --var=TJ results/gabls-3
```

For users of older SCM versions, the following steps are applicable.

```
ouv_exp_scm
r.make_exp
linkit
cp ${SCM_MODEL_DFILES}/run_configs/gabls-3/* .
```

```
runscm
tsplot --var=TJ results/gabls-3
```

Generating SCM Input Data

The SCM accepts input data from two distinctly different sources for different applications. In column-model intercomparison projects such as GABLS-3, a set of prescribed forcing fields are defined as inputs. In this case special project data that includes advective tendencies, vertical motion and other terms is provided by the project organizers. The SCM can also be run using data generated by GEM integrations, with forcing terms either diagnosed or emitted directly from the GEM model itself. To initialize with model data, a preprocessing step is required to perform the necessary forcing diagnostics and prepare the input files for the SCM.

Input for the SCM can be broken into two broad categories: dynamics input and physics input. "Dynamics Input" consists of initializing fields (i.e. temperature, wind, humidity profiles and surface pressure) and forcing fields (advective tendencies, adiabatic expansion, etc). This input must be provided to the SCM in the `ptxt` format described below. Tools are provided with the SCM to ease the process of generating these files. "Physics Inputs" consists of geophysical fields, surface analyses and climatological fields, all of which are read within the physics package itself. The physics reading mechanism is designed to work for RPN Standard Files, which must be provided to the SCM in addition to the dynamics inputs. A set of such files for the GABLS-3 case is included with the SCM package in the `$SCM_MODEL_DATA/dfiles/gabls-3` directory. In practice, the analysis file will usually consist of a model-coordinate analysis, the geophysical file will usually the result of a GenPhysX command, and the climatological file will be obtained from an operational archive.

Special Project Data

Special project dynamics input data must be entered by hand since it can appear in any format (i.e. an email description, a webpage, a pdf document, etc). An example of such data for the GABLS-3 project is provided with the SCM in the `$SCM_MODEL_DATA/dfiles/gabls-3` directory. Manual entry of this kind of data requires an intimate knowledge of the `ptxt` file format since headers and data segments must be generated for each update interval.

The physics input data for special projects will likely take the form of a mix of analyzed fields and fields prescribed by the project organizers (for example, they may prescribe the surface roughness while allowing each modelling group to choose its own fractional clay content). As a first step, a full set of physics input files (analysis, climatology and geophysical) should be generated for the appropriate date. If more recent data is to be used for an older case study, the datestamp modifier contained in the SCM package can be used to redefine the validity date of data in a file. Once the physics input files have been obtained, fields within them can be set to prescribed values using the field prescription tool also provided with the SCM. This tool allows the user to define a constant value

for any field in an input file. For example, the roughness length in the geophysical input file could be set to exactly 0.2 m to conform with the project description. This allows the physics input files to be easily tailored for the special project application.

Initializing from Model Output

Initializing the SCM with data from GEM allows users to reproduce the results of a full 3D GEM integration in a single column. Of course, this approach to developing and testing physical parameterizations has its limitations since the forcing terms that apply to the dynamical equations of the SCM are prescribed by the driving model. However, feedback from physical tendencies is active, allowing for different evolution of the profile depending on the details of the physical schemes applied.

The `prepscm` preprocessor is provided to ease the process of preparing the dynamics input files for the SCM. The preprocessor performs several important tasks on the input RPN Standard Files:

- computation of dynamical forcing terms (horizontal advections, ageostrophic accelerations, etc)
- horizontal interpolation and vector rotation to a geographical (v-direction N/S, u-direction E/W) coordinate system at the profile point
- creation of a separate `ptxt`-formatted SCM input file for each available valid time

The forcing terms are either diagnosed or prescribed, depending on the presence of direct model forcings in the input files. In either case, the following variables must appear in the input files: P0, TT, UU, VV, WW, GZ, HU. Tracer fields may also be prepared for the SCM using the `--tracers` argument to `prepscm`. The SCM relies on the vertical grid descriptor (!!) record for its interpretation of the vertical coordinate in the input RPN Standard File. This record may not exist in older files, but may be created with the `add_toctoc` application distributed with the vertical grid descriptor package.

Direct Model Forcings

The `prepscm` preprocessor is capable of computing the forcing terms required by the SCM (advection, geostrophic wind, etc). These calculations provide results that are generally close to those produced by the GEM model, especially in regions where terrain is relatively flat. However, for the most accurate SCM simulation possible, it is preferable to have the dynamical forcings prescribed by requesting these outputs in the GEM model output configuration:

GEM Output Name	Package	Description
TADV	Physics	Dynamics tendency for temperature (advection and solver)
TTND[1]	Dynamics	Dynamics tendency for temperature (advection, solver and diffusion)
UADV	Physics	Dynamics tendency for u-component wind (advection and solver)
UTND[1]	Dynamics	Dynamics tendency for u-component wind (advection, solver, diffusion and stagger/destagger)
VADV	Physics	Dynamics tendency for v-component wind (advection and solver)
VTND[1]	Dynamics	Dynamics tendency for v-component wind (advection, solver, diffusion and stagger/destagger)
QADV	Physics	Dynamics tendency for specific humidity (advection and solver)

The more frequently these fields are produced by the driving GEM model, the more accurate the forcings for the SCM will be. In cases where the reproduction of the GEM results is very important, these fields should be generated at every time step. Note that the tendencies do not need to be produced for the whole grid, just for a small region (enough to allow for cubic horizontal interpolation) around the chosen profile point for the SCM. Since the "*TND" grid include all non-physics adjustments to the physical world - rather than just those that occur before the physics during the time-step - they are the preferred fields for GEM reproduction purposes. Since tracers are not currently diffused by GEM and are defined at mass points in the Arakawa C-grid, the "QADV" field is sufficient to reproduce the full dynamics tendencies on specific humidity.

At the lowest SCM level (the "diagnostic" level) close to the surface, GEM dynamics might reasonably not be expected to create any tendencies on the physical world variables since this level is intended for the physics only. However, the stagger/destagger operation causes increments between the physics tendency application (unstaggered) and the "physical world" update following diffusion that introduces non-zero tendencies at the bottom level of the "UTND" and "VTND" grids. This effect may be noticeable, especially on coarse GEM grids.

The physics inputs for GEM input data will likely be identical to those used by the GEM model to generate the forcing fields. The climatological, geophysical and analysis input files should be provided to the SCM at run time.

The PTXT File Format

The SCM uses the simple `ptxt` (profile text) input file format. This format was chosen because it allows for manual validation and modification of the input values if required. A `ptxt` file consist of two main components: a header, and a data segment. An example of the current (version 1) `ptxt` header format for GABLS-3 data is described here.

```
PTXT 1 4
step 4 51.971 4.9267 -0.7 nearest 20060701.200000
1 PRE_P0
7 PRE_WW ADV_TT ADV_HU ADV_UU ADV_VV GEO_UU GEO_VV
```

- Line 1: File is in `ptxt` format, version 1, with 4 lines in the header.
- Line 2: New tendencies are applied immediately (step), are of kind 4 (height), for a column at 51.971N, 4.9267E, elevation of -0.7m, with `nearest` horizontal interpolation used to define field values, and are valid at 2000 UTC 1 July 2006.
- Line 3: 1 surface field is provided, a prescribed (PRE) surface pressure (P0).
- Line 4: 7 upper-air fields are provided, advective (ADV) tendencies of temperature, humidity and winds, and geostrophic (GEO) values of the wind components.

The remainder of the file (the data segment) consists of a set of lines that describe the forcing data. There is one line for field name identified in the header, and they must appear in the same order as they are listed in the header. For example, the temperature advection tendency term for 2000 UTC 1 July 2006 in GABLS-3 is described here.

```
ADV_TT 5 T linear 30000 1500 1000 200 0 0. 0. -2.5E-5 -2.5E-5 0.
```

- Line 1: Temperature advection (ADV_TT) profile, defined at 5 thermodynamic (T) levels, with linear vertical interpolation to be used on the profile. Values of the profile are provided at 30000 m, 1500 m, 1000 m, 200 m and 0 m; the values are 0, 0, -2.5E-5, -2.5E-5 and 0.

Note that mks units are used throughout the SCM, so the advective tendencies should be provided in K/s.

Running the SCM

The actions involved in running the SCM have been made as similar to running an interactive version of GEM as possible. A pair of files (`configexp.cfg` and `scm_settings.nml`) and a run script (`runscm`) are all that is required to run the SCM once the working directory has been established.

SCM Configuration Files

The `configexp.cfg` file is optional, and allows you to retain a copy of the input arguments sent to the `runscm` script. For example, the GABLS-3 data can be obtained using the following configuration file.

```
SCM_inrep=${SCM_MODEL_DFILES}/dfiles/gabls-3;                                  #Directory containing .ptxt files from
prepscm
SCM_anal=${SCM_MODEL_DFILES}/dfiles/gabls-3/analysis/2006070112_000;           #Inializing FST analysis (surface fields)
SCM_climato=${SCM_MODEL_DFILES}/dfiles/gabls-3/climato/clim.gabls;             #Climatological FST file
SCM_geophy=${SCM_MODEL_DFILES}/dfiles/gabls-3/geophy/geo.gabls;                #Geophysical FST file
SCM_ozone=${ATM_MODEL_DFILES}/datafiles/constants/ozoclim_Fortuin_Kelder1998; #Ozone climatology
SCM_radtab=${ATM_MODEL_DFILES}/datafiles/constants/irtab5_std;                 #Radiation tables
SCM_const=${ATM_MODEL_DFILES}/datafiles/constants/thermoconsts;                #Constants file
```

The environment variables `${SCM_MODEL_DFILES}` and `${ATM_MODEL_DFILES}` are defined when the SCM SSM package is loaded. Each line of this file corresponds to an optional argument that can be provided to the `runscm` script that is used to launch the SCM. The configuration file can be absent, or may contain only a subset of the possible list of arguments. In this case, either the argument values (if provided) or package defaults (if no argument is provided) will be used. Maintaining a `configexp.cfg` is therefore a good idea for bookkeeping at least.

The `scm_settings.nml` file contains a set of namelists that are used by the SCM. For detailed information, see the latest SCM namelist documentation, The `&scm_cfgs` namelist contains a description of the SCM model configuration, including vertical coordinate and output information, while the `&step` namelist (common with GEM) contains information about model start time, step length and number, and update interval. An optional `&phydata` namelist allows for specification of additional physics input data for the phydata package. A list of namelist keys for older releases can be accessed in the `&scm_cfgs` archive. The other namelist contained in `scm_settings.nml` is the `&physics` namelist. Since the RPN Physics package is responsible for reading its own namelist, the physics documentation for the appropriate physics version should be consulted for details about the available keys.

Loading Canonical Case Data

The SCM comes with a set of canonical (or "pre-canned") cases that provide easy access to simulations in a range of environments. These cases have been selected for any one of a number of reasons, including:

- The availability of high quality verifying data.
- The simplicity of idealized state and forcing profiles for a specific type of airmass or region.
- Their ability to illustrate particular behaviours in the RPN Physics.

The complete list of the currently available canonical cases and short case descriptions can be obtained interactively.

```
scmcase -list
```

To load a case into the current SCM experiment, use the `-get` option to `scmcase`.

```
scmcase -get gabls-3
```

In this example, the GABLS-3 case configuration is extracted into the local directory. This extraction will prompt before overwriting existing case configuration files (generally `configexp.cfg` and `scm_settings.nml`). To overwrite existing case configurations without prompting, use the `-h` argument to `scmcase`. A list of additional command line arguments can be obtained.

```
scmcase -h
```

If you are studying an interesting case, consider preparing it as a canonical case for inclusion in future SCM releases.

MJO-AMIE Case Description

This three-month integration includes both active and suppressed phases of the Madden-Julian oscillation (MJO; Madden and Julian (1971; 1972)) in the tropical west Pacific and the Indian Oceans. The case is based on special measurements from the the US Department of Energy's Atmospheric Radiation Measurement (ARM) MJO Investigation Experiment (AMIE), taken from October 2011 to March 2012 on Gan Island in the Indian Ocean south of India. Forcings are available every 3hrs from 20111001.000000 to 20111231.210000. Simulation can start from any 000UTC date during this period. This case may be particularly useful for investigation of tropical convection and cloud properties in the RPN physics package.

Detailed case documentation provides additional details about the model configuration and expected results.

Launching the SCM

Once the configuration files have been created or modified, the SCM can be launched using the `runscm` command. A list of optional command line arguments can be obtained.

```
runscm -h
```

As noted above, arguments can be provided to this script, or their values can be inserted into the `configexp.cfg` file. The `runscm` command will run the SCM interactively (the SCM cannot be submitted in batch mode since integration times tend to be on the order of a few seconds to a minute) and produce outputs in `RUNSCM/output`. The model listing appears on the screen, but can be redirected to a file (for example, `scm_listing.out`).

```
runscm >scm_listing.out
```

This is particularly useful when establishing the list of SCM model outputs since the contents of the buses are displayed near the top of the model listing.

Specifying Output Fields

Unlike the GEM model, the SCM does not support an `outcfg.out` file. The parsing and interpretation of that file is complex, and adds little to the relatively simple SCM. Similarly, a parallel to the output processing package of GEM has not been implement in the SCM in order to maintain simplicity. Instead, users have access to all values on the buses that are passed between the SCM dynamics and the RPN Physics package being run. For users familiar with the GMM package, keys defined as GMM names are also available for output; however, this form of output request is not likely to be heavily used and is subject to mangling because of special characters used in some GMM names.

Outputs are requested in the `&scm_cfgs` namelist using the `output_list` key. By default, fields are written at every timestep during the SCM integration. This behaviour can be changed using any combination of the `output_start`, `output_end` and `output_inc` (increment) keys also available in the `&scm_cfgs` namelist. These keys apply to all items in the output list. Since the quantity of output data is limited because of the single model point, it is likely that most SCM integrations will produce output at all timesteps (the default). In the following example, a limited set of output variables are requested at all steps.

```
&scm_cfgs
output_list = '2T','FC','TJ',
...
/
```

The temperature (2T, see the discussion of output naming below), sensible heat fluxes (FC) and diagnostic-level temperature (TJ) are written into separate output files in the `RUNSCM/output/series` directory. For all SCM integrations, vertical coordinate information in height, pressure and sigma-like units are also written in the `RUNSCM/output/coord` directory. These outputs are needed for post-processing involving upper-air values, including profile plots.

The output file format is described in more detail in the section of this guide devoted to output handling. However, it is important to note here that all outputs are written in text format and are therefore subject to Fortran formatting. While the output system attempts to estimate the best format to use to write the values based on the range of the field, there may be times where a different output format is required. In this case, the Fortran-style format should be specified in parentheses following the variable name.

```
&scm_cfgs
output_list = '2T(f16.10)','FC','TJ',
...
/
```

In this example, the temperature field would be written to the output file in 16 digits total (including the decimal point), with 10 following the decimal point.

Specific levels for some variables may also be requested (be default, all levels of a variable are produced). This is accomplished using a colon (:) separator in the variable name.

```
&scm_cfgs
output_list = '2T','FC','TJ:5',
...
/
```

In this example, only the 5th-level sensible heat flux (aggregated value) is written to the output file. If both a specific level and a specified output format are required, then the level information should be provided first.

Output Field Names in the SCM

A complete list of the items in the SCM buses is displayed in the run-time listings of the SCM. It is often easiest to run once with no `outout_list` specified in the `&scm_cfgs` namelist of the `scm_settings.nml`, then look at the SCM bus listings to decide which fields to request. The bus listings appear near the top of the listing file. An example of a bus entry listing is shown here for the temperature on the dynamics bus.

```
| PW_TT:P        |   "2T  "   | TEMPERATURE AT T+DT                  |    244 |     81 |   1 |   0 | 1 | 0  |
```

In the second column (`"2T "`), the output name is defined. The third column contains a brief description of the variable. In order to obtain temperature information from the SCM, output of `2T` should be requested. Note that this is somewhat different from GEM, where certain variables are given more common aliases. For example, `TT` is used for temperature outputs rather than `2T`, and conversion from Kelvin to Celsius is also performed. Output from the buses in the SCM is blind: bus output names must be used, and no unit conversions are performed. A list of some common output requests is provided for reference

Standard File Name	SCM Output Name	Description	SCM Units
TT	2T	Dry bulb temperature	K
UU	UP	Westerly wind component	$m\ s^{-1}$
VV	VP	Southerly wind component	$m\ s^{-1}$
WW	OM	Pressure coordinate vertical motion	$Pa\ s^{-1}$
GZ	GM	Geopotential (g x GZ) above ground level	$m^2\ s^{-2}$
HU	UH	Specific humidity	$kg\ kg^{-1}$

GMM Name Mangling

Some GMM variable names (keys) contain special characters that must be replaced before an output file prefixed with the variable name can be produced. In the `outout_list` request, specified in the `&scm_cfgs` namelist of the `scm_settings.nml`, GMM names should be used directly. Matching for GMM output is done using the requested names. For output, however, the SCM mangles the requested name if special characters are found. For example, `TR/HU:P,W` is the GMM key for specific humidity (`HU` in standard encoding); however, since the slash (/) character is interpreted as a Unix directory break and the colon (:) character is used as a separation between a host and a path, these characters must be replaced with delimiters that are do not have special significance for the shell. The SCM replaces all slash (/) characters in GMM keys with an underscore (_), and all colons (:) with a dash (-). Thus, the GMM key for specific humidity in this example becomes `TR_HU-P,W` for output file naming.

Understanding SCM Output

Output from the SCM is stored under the `RUNSCM/output` directory. Two subdirectories are created for each SCM integration: `coord/` contains a set of files describing the vertical structure of the profile; and `series/` contains data files corresponding to the output fields requested in the `&scm_cfgs` namelist. Each field (variable) is written in a separate output file that is prefixed with the variable name. For example, the file `RUNSCM/output/FC_20060701.120000.txt` for the benchmark GABLS-3 is shown here.

```
date              1                2                3                4                5
20060701T120000Z 0.4254754500E+01 0.0000000000E+00 0.0000000000E+00 0.0000000000E+00 0.4254754500E+01
20060701T120730Z 0.8137049900E+02 0.0000000000E+00 0.0000000000E+00 0.0000000000E+00 0.8137049900E+02
20060701T121500Z 0.1083913900E+03 0.0000000000E+00 0.0000000000E+00 0.0000000000E+00 0.1083913900E+03
```

The first row contains a header describing the columns. The first column of each file contains the valid date/time of the entry using the ISO 8601 Basic format. Handling this format is described in more detail in the context of output file manipulation later in this guide. The increasing level number for the variable is specified in the header following the "date" entry. Levels are defined as in the SCM and

RPN Physics package, where surface fields (such as FC in the example above) have five values for the four different surface types and aggregation, while full profile fields (such as 2T, the profile temperature) begin with the first index (1) at the top of the column. Currently, there there is no distinction made between fields that are defined on thermodynamic levels and those defined on momentum levels. It is hoped that future versions of the RPN Physics package will allow for the addition of this information to the output file names to ease post-processing.

On-board Display Tools

The SCM package provides a set of minimal tools for displaying output data. Both take the form of Python programs that call R scripts. Time series of a field at a specified level can be plotted with `tsplot` while instantaneous profiles can be plotted with `profplot`. Both programs accept a `--help` argument that displays usage information. By default, the ImageMagick `display` program is used to present the plot after its creation; however the graphical display tool can be changed using the `--show` argument to the program, or immediate presentation can be suppressed with the `--batch` option. The default graphical display tool can be user-configured by setting the value of `display_application` (a blank value makes batch behaviour the default).

Plotting Time Series Data

A time series for any level of a given field can be plotted with `tsplot`. Any number of SCM experiments can be compared on the same plot by providing the path the the appropriate outputs as arguments to the `tsplot` utility (this path is displayed at the end of the SCM integration and consists of the `SCM_xfer/SCM_exp` from the run configuration). The name of the variable to plot must be provided as the value to the `--var` argument. Note that GMM variable names may be subject to mangling. By default, the first level (index=1) is plotted unless a `--level` argument is provided. The default output is a PNG plot, although a JPEG or PostScript plot can also be generated using the `--device` argument.

```
tsplot --var=TJ --level=1 --device=png results/gabls-3
```

In this example, a time series of the near-surface temperature (TJ) is generated as a PNG plot as shown in the QuickStart.

```
tsplot --var=2T --level=40 results/gabls-3
```

In this example, a time series of the SCM level 40 (from the top) dry bulb temperature (2T, see the discussion of output field naming above) is generated as a PNG plot.

```
tsplot --var=FC --level=5 --device=jpeg results/gabls-3 results/new_test
```

In this example, time series of the aggregated sensible heat flux from a pair of tests writing into the `results/` subdirectory (named `gabls-3` and `new_test`) are compared in JPEG plot.

Plotting Profile Data

A profile of a field can be plotted in pressure coordinates at a given time using the `profplot` program. Any number of SCM experiments can be compared on the same plot by providing the path the the appropriate outputs as arguments to the `profplot` utility (this path is displayed at the end of the SCM integration and consists of the `SCM_xfer/SCM_exp` from the run configuration). The names of the variables to plot must be provided as the value to the `--var` argument. Multiple variables can be specified so that they are plotted on the same panel. Note that GMM variable names may be subject to mangling. The forecast time to plot should be provided using the `--time` argument. Any of hours (`h`), minutes (`m`) and seconds (`s`) can be provided. For example, `--time=2h30min` will produce a profile of the profile after 2.5h of integration (assuming that this is an output time interval of the SCM). A value of `--time=12h` as shown in the example below will result in a profile of the 12h forecast profile. Multiple times can be plotted using a start:end:step notation: `-t 0h:12h:6h` plots six-hourly data from 0h to 12h into the integration. By default, fields are assumed to be placed on thermodynamic levels: the `--levtype` argument can be used to specify the level type explicitly. A top (lowest pressure) level for the plot can be specified using the optional `--top` argument. The default output is a PNG plot, although a JPEG or PostScript plot can also be generated using the `--device` argument.

```
profplot --var=2T --time=12h --top=500 results/gabls-3 results/new_test
```

In this example the 12h forecast profile of dry bulb temperature (2T, see the discussion of output field naming above) from two different integrations (stored in `results/gabls-3/` and `results/new_test`) are plotted from the surface to 500 hPa as PNG plot.

```
profplot --var=2T --time=1h30m results/*
```

In this example, the 1h 30min forecast profiles of dry bulb temperature from all SCM outputs contained under the `results/` subdirectory are plotted together. Note that wildcard expansion occurs in the shell and will therefore catch any matching paths regardless of whether or not they contain SCM output.

Plotting Tephigrams

A tephigram for SCM model output can be plotted using the `tephiplot` utility. Any number of experiments can be compared on the same plot by providing the path the the appropriate outputs as arguments to the `tephiplot` utility (this path is displayed at the end of the SCM integration and consists of the `SCM_xfer/SCM_exp` from the run configuration). By default, the time-plus state is

plotted, but this can be changed by providing appropriate arguments (run `tephiplot -h` for details). Other components of the basic interface are very similar to those of the `profplot` interface described above.

```
tephiplot -t 6h:12h results/gabls-3
```

In this example, a tephigram is plotted with the 6h and 12h forecast profiles shown.

On-board Diagnostic Tools

A minmal, yet extenstible, set of diagnostics are available in versions 1.5.a8.1 and beyond of the SCM. Diagnostics are written into the standard SCM output directory structure so that they are available for plotting with the standard display tools. Additional diagnostics developed by users that may be of more general use should be incorporated in future versions of the package.

Basic SCM Diagnostics

The `scmdiag` tool provides a framework for exensible SCM diagnostic system. While the utility itself performs no calculations, it provides access to both on-board (i.e. released with the SCM) and user-defined plugins that provide diagnostic functionality. Diagnostics can be computed for any number of SCM experiments at the same time, with the output of each provided as an argument to the `scmdiag` utility. For a complete list of the available default functionality, the `--list` option is available.

```
scmdiag --list
```

The variables that can be computed are displayed for user selection, along with a brief description and information about the required inputs. Diagnostic fields are selected using the `--var` argument, a comma-separated list of the fields to be computed. In this example, the wind speed and direction are computed for the gabls-3 benchmark case.

```
scmdiag --var=WSPD,WDIR results/gabls-3
```

The diagnostic values appear as additional outputs in the SCM output directory so that they are immediately available for plotting via the standard display tools. Extra plugins can be employed for additional functionality with the `--plugin` argument (presented as a comma-separated list). The location of these additional plugins can either be defined using a full path in the `--plugin` argument, or via the `SCMDIAG_PLUGIN_PATH` environment variable (colon-separated as is standard for path variables).

Diagnostic Plugins

The basic SCM diagnostics provided with the package are relatively minimal. However, the diagnostic plugin system allows for any level of user extension required. A plugin is a small package that contains the elements of a rudimentary interface between the `scmdiag` utility and user-generated diagnostic scripts. The calculations can be performed in any language (R, python, octave, et cetera), although proprietary packages such as matlab and IDL should be avoided if possible because licensing issues may make their general use more difficult. The top level of a diagnostic plugin is a directory with the name of the plugin and the `.sdp` extension (SCM diagnostic plugin). Within this directory, two components are mandatory:

- A world-executable file named `exec` that will be executed to run the plugin diagnostic
- A documentation file name `doc` that contains information about the available diagnostic fields (the names of the diagnostics must appear first on each line, followed by description)

A third component is standard but not required:

- A `src/` subdirectory that contains the scripting that implements the diagnostic (called by the command in the code>exec file)

An example of a simple diagonstic plugin is available at `${SCMDIAG_PLUGIN_PATH}/winds.sdp`

More information about developing your own plugin is available in the technical documentation.

Handling Outputs Directly

Output from the SCM is written into output text files whose structure is designed to be easily ingested by a range of mathematical and plotting applications. This allows users to readily manipulate and display data produced by the SCM. Both R and MATLAB are capable of reading the SCM output tables directly.

An Application Program Interface (API) has been developed for R scripts, the description of which is available in the Technical Documentation. For users that do not wish to use the API, the following statements in R read the SCM output file for surface fluxes into a data frame and computes the Bowen ratio.

```
flux.sensible<-read.table('RUNMOD/output/series/FC_20060701.120000.txt',header=TRUE)
flux.latent<-read.table('RUNMOD/output/series/FC_20060701.120000.txt',header=TRUE)
```

```
bowen<-flux.sensible/flux.latent
```

Physics Interoperability

Interoperability between the SCM and the RPN Physics package is limited only by changes to the Physics API. Incrementation of a physics compatibility number (PCN) indicates a change in this API and will limit interoperability. This table lists the known interoperability of the SCM and available physics packages. The SCM is primarily used with the RPN Physics package; however, for intercomparison purposes running with the CCCMa physics package is also possible.

SCM Version	Compatibility Number	Interoperable Physics		Corresponding GEM Bundles	Notes
		RPN Physics	CCCMa Physics		
0.0.0-b1	-	5.2.1		4.2.0-b2	Physics compatibility number not yet implemented.
0.0.0-b2	-	5.2.1		4.2.0-b2	Physics compatibility number not yet implemented.
1.0.0	-	5.2.1		4.2.0	Physics compatibility number not yet implemented.
1.0.1	-	5.2.1		4.2.0	Physics compatibility number not yet implemented.
1.0.2	-	5.2.1		4.2.0	Physics compatibility number not yet implemented.
1.0.3	-	5.2.1		4.2.0	Physics compatibility number not yet implemented.
1.0.4	-	5.2.2		4.2.1	Physics compatibility number not yet implemented.
1.0.5	-	5.2.2		4.2.1	Physics compatibility number not yet implemented.
1.0.6	-	5.2.2		4.2.1	Physics compatibility number not yet implemented.
1.2.0-a1	1	5.4.0-b9		4.4.0-b9	Initial release following major upgrades to the physics interface.
1.2.0	1	5.4.7		4.4.5	
1.4.0-a1	6	5.7.0-a9		4.7.0-a9	Preliminary stabilization of the new physics interface.
1.4.0-a2	6	5.7.0-a9		4.7.0-a9	
1.4.0-b2	6	5.7.0-b2		4.7.0-b2	
1.4.0-b3	6	5.7.0-b3		4.7.0-b4	
1.4.0-b4	6	5.7.0-b9		4.7.0-b9	Minor adjustments to dynamics bus entries and interface for `phy_init()`
1.4.0-rc1	6	5.7.0-rc3		4.7.0-rc3	
1.4.0-rc2	6	5.7.0-rc4		4.7.0-rc4	
1.5-a1	6	5.8-a1		4.8-a1	Minor adjustments to whiteboard entries crossing the interface
1.5-a2	6	5.8-a2		4.8-a2	
1.5.a3	7	5.8.a3		4.8.a3	
1.5.a5	7	5.8.a5		4.8.a5	
1.5.a6	7	5.8.a6		4.8.a6	
1.5.a7	7	5.8.a7		4.8.a7	
1.5.a8	7	5.8.a8		4.8.a8	

1.5.a10	7	5.8.a10		4.8.a10	
1.5.a11	7*	5.8.a11		4.8.a11	Physics tendency mask now ranges from [0-1] instead of [0-dt] (physics compatility number will be increase in the subsequent release)
1.5.b2	9	5.8.b2		4.8.b2	Backwards-compatible augmentation of the Physics API
1.5.b6	9	5.8.b6		4.8.b6	
1.5.b7	9	5.8.b7		4.8.b7	
1.5.rc3	12	5.8.rc2		4.8.rc3	Rollback of recent API changes unrelated to SCM
1.5.rc8	12	5.8.rc8		4.8.rc8	
1.5.rc10	12	5.8.rc10	5.8.rc10	4.8.rc10	Export ptop via whiteboard to complete physics API
2.0.a1	12	6.0.a1		None	
2.0.a2	12	6.0.a2		None	
2.0.a4	12	6.0.a4		None	
2.0.a5	12	6.0.a5		None	
2.0.a13	12	6.0.a13		None	
2.0.a17	12	6.0.a17		5.0.a9	Provide list of requested output fields to the physics as API enhancement
2.0.a19	14	6.0.a19		5.0.a11	Additional grids passed to phy_init
2.0.a20	14	6.0.a20		5.0.a12	None
2.0.b4	14	6.0.b4		5.0.b4	None
2.0.b6	15	6.0.b6		5.0.b6	Add handshaking for horizontal operations (smoothing) requested by the physics, implemented as a copy in the SCM.
2.0.b7	15	6.0.b7		5.0.b7	The change to the integral equation solver (Bug 9503) leads to small changes in the surface temperature in the GABLS-3 and other benchmarks.

Running with CCCMa Physics

The SCM is intended primarily for use with the RPN Physics pacakge, but the Physics API is designed to accommodate any model physics as transparently as possible once the physics-side API has been implemented. As a result, it is also possible to connect the SCM to the CCCMa physics package for testing and intercomparison. Although the SCM supports CCCMa physics, it is up to the user to provide the additional surface analysis and geophysical fields required by the parameterizations used in that package.

Because switching physics packages requires modifications to the development environment, any single experiment should use only one physics package. Setting up a CCCMa physics-based experiment is relatively simple, and includes a step in which the SCM executable is relinked to the CCCMa physics library. These instructions assume that a CCCMa-interoperable of the SCM has been loaded by the user via the standard `s.ssmuse.dot` command as described in the quickstart guide. In this example, the experiment `MY_EXP` should be replaced with the experiment name of your choice, as should the version of the CCCMa physics to be acquired.

On the Dorval network, the initialization required to run CCCMa physics is simple.

```
. r.load.dot SCM/1.5.3
. scmdev.dot MY_EXP -update ENV/x/cccmaphy/5.8.rc10 -cd
cp ~armnrmt/scm_cccma/cfgs/* .
```

On the Science network, the initialization changes slightly.

```
. r.load.dot SCM/1.5.3
. scmdev.dot MY_EXP -update eccc/mrd/rpn/MIG/ENV/x/cccmaphy/5.8-LTS.10 -cd
cp /home/rmt001/local/eccc-ppp2/data/scm_cccma/cfgs/* .
```

On both networks, the instructions to run the SCM with CCCMa physics (following one of the two initializations above) and analyze the results with the standard SCM analysis and display tools are identical.

```
make -j obj
make scm
runscm
tsplot -v SFCT results/cccmaphy/ -g topright
```

Additional Features and Tools

Some additional utilities and support features are provided along with the column model in the SCM package. These are applications and addons that will be of use to users working with the SCM and associated model physics. Other external tools that may also be useful include:

- Create a well-constructed set of hybrid coordinate levels with `Um_make_levels` (a GEM package of v4.1.4 or newer must be loaded for access to this utility).

Application Configuration

To allow a user to set preferences for the behaviour of different applications surrounding the column model, a user configuration file is read at the start-up of configurable utilities. The default configuration file can be used as a template, and is available at `$scmscripts/default_config` for copying to your home directory as `.scmrc`. Comments in the default configuration file provide guidance for setting application-specific or default options.

Entry Program (prepscm)

A set of input data files containing information about the atmospheric state of the profile and dynamical forcings must be provided to the SCM using the `SCM_inrep` argument to `runscm`. These files can be generated using the `prepscm` preprocessor.

There are two fundamental types of SCM input data: special project data and model output data. Special project data must be entered by hand using the PTXT file format. The generation of PTXT files from RPN Standard (FST) files (usually produced by a GEM integration is accomplished using the `prepscm` preprocessor. Currently, this utility has only been tested with model-coordinate FST files as inputs; however, it should generate column information using pressure-coordinate files as well. Regardless of the vertical coordinate of the FST input, the "toc-toc" (!!) record must be present. When using older files that do not contain this record, the `add_toctoc` utility should be used to add the required field to each of the input files before `prepscm` is run. The PTXT files generated by `prepscm` are created in the directory from which the `prepscm` command is executed. If the input files exist in a directory that is not writeable by the user, then a path prefix can be added to the input file names.

The `prepscm` utility extracts column information from the FST files provided and computes a set of forcing fields for the SCM. To do this, the application needs to be given information about the profile position and interpolation strategy. Usage information and a list of available options can be obtained for `prepscm`.

```
prepscm --help
```

The standard calling sequence is `prepscm [options] [file(s)]`, where `[file(s)]` is a file -- or list of files -- that are to be preprocessed into PTXT files. This list can contain wildcard characters so that many files can be matched quickly. For long lists of input files that exceed the shell line length limit, wildcards can be quoted to avoid shell expansion. Expansion is then done internally so that the full list of matching input files is retrieved.

The `--lat` and `--lon` arguments position the profile on the grid. Extrapolation of limited-area grids to the profile point is not allowed. The `--interp` argument can be used to specify the horizontal interpolation scheme to be used to compute the column state and forcings (default is cubic horizontal interpolation). If tracer fields are available, then they can be specified using the `--tracers` argument. The value for this argument consists of a comma-separated list of tracer fields to be interpolated to the profile point and evaluated for advective forcings throughout the integration. The default value for `--tracers` is `HU`, which ensures that specific humidity is included in the SCM initialization and evolution. An optional argument (`--log`) specifying a log file to write is provided as an alternative to standard output.

The state of the column (profiles of the basic variables) is required only at the initial time; thereafter, forcing fields representing the effects of advection, geostrophic adjustment, adiabatic expansion, and others are prepared for SCM input. This means that the desired initial time of the SCM integration must be known by the preprocessor so that the appropriate state can be generated. This state appears in a special PTXT file that is prefixed with `init_` rather than the standard `forcings_`, which serve for the remainder of the inputs to the SCM. The FST file containing the initial conditions should be specified with the `--init` argument to `prepscm`. If this file contains multiple valid times, then an initialization file is generated for each valid time.

Examples

In this example, output from a GEM integration has been saved in the users' current directory (model-coordinate, FST-formatted with the toc-toc record already present). The user wishes to reproduce the results of the GEM integration as closely as possible, at the gridpoint nearest to 61.4N, 140.8W, and therefore choses to use "nearest" neighbour horizontal interpolation. The SCM will be run starting from the 24h GEM forecast in this case, and all available data from this run will be used to compute forcing fields.

```
prepscm --lat=61.4 --lon=219.2 --interp=nearest --init=2009011512_024 2009011512_???
```

- [`--lat=61.4`] positions the profile point at 61.4N.
- [`--lon=219.2`] positions the profile point at 219.2E (140.8W, but all longitudes should be entered as positive values in the range 0-360).
- [`--interp=nearest`] tells `prepscm` to use the nearest gridpoint rather than interpolating to the precise position specified.
- [`--init=2009011512_024`] specifies the 24h forecast as the initializing data so that the profile state (a PTXT file prefixed with `init_`) is generated from the data contained in this file.
- [`2009011512_???`] provides a shell-expanded list of files matching all forecast hours for this run. All matching files in the directory will be treated to create a corresponding set of PTXT forcing files prefixed with `forcings_`.

In this example, output from a long GEM integration is available in the directory of another user (model-coordinate, FST-formatted with the toc-toc record already present). The default cubic interpolation is to be used to generate a profile that most closely matches the atmospheric state at the

precise point identified (45.5N, 7.5E). The SCM will be started from the 12h forecast of the GEM integration and will use all matching files to compute the forcing fields. The cloud water mixing ration (QC) will be both initialized and forced over the course of the SCM integration.

```
prepscm --lat=45.5N --lon=7.5 --init=/path/to/files/from/other/user/2010092300_012 --log=prep.log
'/path/to/files/from/other/user/2010*'
```

- [`--lat=61.4`] positions the profile point at 45.5N.
- [`--lon=219.2`] positions the profile point at 7.5E.
- [`--init=/path/to/files/from/other/user/2010092300_012`] specifies the 12h forecast as the initializing data so that the profile state (a PTXT file prefixed with `init_`) is generated from the data contained in this file.
- [`--log=prep.log`] specifies that all output from `prepscm` should be redirected to a file named `prep.log`
- [`'/path/to/files/from/other/user/2010*'`] identifies a -- possibly very long -- set of input files to treat to generate PTXT forcing files prefixed with `forcings_`. To avoid possible problems with line length on shell expansion, the string specifying the file list is quoted so that expansion occurs within the `prepscm` application.

Datestamp Modifier (forcedate)

When generating input data for the SCM, it may be useful to change the validity date/time stamp of a record in an RPN Standard File (FST). This is particularly true for special project data when analysis fields at the correct dates and/or times may not be available. The rudimentary `forcedate` utility provided with the SCM package allows a user to change validity date information in a way that is more complete than what's available when using `editfst`.

The command line interface to `forcedate` is positional, and can be printed by providing an insufficient number of arguments.

```
forcedate
```

The result is an error message that identifies the expected arguments.

```
CRITICAL ERROR: Usage: forcedate INPUT OUTPUT FIELD_NAME DATE DEET NPAS
```

- `INPUT` is the input file name.
- `OUTPUT` is the output file name.
- `FIELD_NAME` is the name of the FST record whose dates/time information will be modified (only a single field name is allowed, but all levels of that field are treated).
- `DATE` is the FST date stamp to apply (dateo). This can be generated from readable year-month-day information using the `r.date` utility.
- `DEET` is the time-step (in seconds) to apply to the field (deet).

- `NPAS` is the number of steps (npas) that will be used in combination with deet to compute the valid time for the field.

Validity time in an FST file is computed as the initial time (dateo) plus the timestep (deet) times the number of steps (npas). Each of these elements of the record associated with a specified field can be set using `forcedate`. Although the `ip2` element gives a useful estimate of the valid hour, it is not the "official" container of validity time, and cannot represent sub-hourly intervals.

Examples

In this example taken from the preparation of the GABLS-3 benchmark case, the sea ice temperature field (I7) is set to be valid at 1200 UTC 1 July 2006 (the initialization time for participating models).

```
forcedate 2006070112_000 2006070112_000.i7 I7 332255600 0 0
```

- [`2006070112_000`] is the name of the FST input file containing the I7 field.
- [`2006070112_000.i7`] is the name of the FST output file to produce with the modified I7 record.
- [`I7`] is the name of the FST record whose date/time information will be modified.
- [`332255600`] is the FST date stamp for 1200 UTC 1 July 2006.
- [`0`] forces the time step element of the record to 0s.
- [`0`] forces the number of steps taken to compute validity time to 0.

Field Prescription (setfld)

Special projects involving column models often involve the specification of many surface quantities. Since geophysical fields are read directly by the physics package from the specified FST-formatted geophysical file (the SCM_geophy argument to `runscm`), the perscription of specific values is accomplished by modifying the geophysical file directly. The `setfld` utility provided by the SCM package allows a user to quickly modify the contents of a field in a FST by specifying a constant value for the record. Any number of fields can be prescribed simultaneously in the same file.

The command line interface to `setfld` is positional, and can be printed by providing an insufficient number of arguments.

```
setfld
```

The result is an error message that identifies the required arguments, which consist simply of the input file from which to read the record and the output file into which to write the identical record with the constant specified value.

```
CRITICAL ERROR: You must specify the input and output files as arguments
```

Since the interface to `setfld` is rudimentary and an unknown number of fields may be specified (from 0 to unlimited), the usage message shown here does not inform the user of the format in which to specify values.

By default, the prescribed value is applied to all levels of the specified field. The level for which to apply a prescribed value can be specified using a colon (:) following the field name. For example, `I0=290.` would prescribe a temperature of 290K for all soil levels; however, an argument given as `I0:1199=290.` would prescribe 290K only for the first-level (skin) temperature. The `setfld` application is blind to the names of the specified fields except for the relationship between Z0 (roughness length) and ZP (log of roughness length used as input for the RPN physics. If either of these fields is specified, then the other is computed and written, along with a message at the end of execution informing the user of this functionality. If both are specified and an inconsistency is found, then `setfld` aborts with an error message.

Examples

In this example taken from the preparation analysis fields for the GABLS-3 benchmark case, a set of analysis fields are set to the values specified in the project description.

```
setfld 2006070112_000 2006070112_000.gabls TM=296.55 I9=270. I0:1199=23.4 I0:1198=295.2
```

- [`2006070112_000`] is the name of the FST input file containing each of the records to be modified.
- [`2006070112_000.gabls`] is the name of the FST output file to produce with the modified fields.
- [`TM=296.55` sets the sea surface temperature to 296.55K.
- [`I9=270` sets the glacier temperature to 270K.
- [`I0:1199=296.55` sets the first-level (ip1=1199) soil temperature to 296.55K.
- [`I0:1198=295.2` sets the second-level (ip1=1198) soil temperature to 295.2K.

In this example taken from the preparation of the geophysical fields for the GABLS-3 benchmark case, a set of geophysical fields are set to the values specified in the project description. The input file is the result of `genphysx` run for a small grid that covers the project profile point.

```
setfld genphysx.fst genphysx.fst.gabls Z0=0.15 VF:1199=0 VF:1198=0 VF:1197=0 VF:1196=0 VF:1195=0 VF:1194=0 VF:1193=0
VF:1192=0 \
VF:1191=0 VF:1190=0 VF:1189=0 VF:1188=0 VF:1187=1.0 VF:1186=0 VF:1185=0 VF:1184=0 VF:1183=0 VF:1182=0 VF:1181=0 VF:1180=0
\
VF:1179=0 VF:1178=0 VF:1177=0 VF:1176=0 VF:1175=0 VF:1174=0 J1=45. J2=25. ME=-.7 MF=-.7 MG=1. VG=13 LH=0. Y7=0. Y8=0.
Y9=0.
```

- [`genphysx.fst`] is the name of the input FST geophysical file generated by `genphysx`.
- [`genphysx.fst.gabls`] is the name of the FST output file to produce with the modified fields.
- [`Z0=0.15`] sets the roughness length to 15cm. The `setfld` consistency check will ensure that ZP, the logarithm of the roughness length, is set to the correct value of -1.9 when the output file is written and will issue a message in this regard.

- [... other arguments continue as in the example above]

Observation Data Processors

Converting observational data to the SCM output format allows for easy comparison of SCM results with observations. Essentially, the observations appear to the SCM plotting tools to be the results of another SCM integration and can be manipulated and displayed as such. Because observational data is stored in many different formats, different data processors are needed for different observational datasets.

ARM Data Processor (arm2scm)

Physics Forcing Data (phydata package)

For specific experiments, the use of direct physics forcing data may be required. For example, an SCM intercomparison project may require that all models be run with a prescribed time-evolving surface skin temperature: this package is designed to help users apply these kinds of conditions. The phydata packge is available only on SCM versions 1.3.0-b2 and higher. The package interface has been kept as simple as possible to ease its integration into projects. Each time that the `pd_get()` function is called (explained in detail below), the requested variable is read and linearly time-interpolated to the current time. To activate the phydata package, all that is required is the addition of an input file in `configexp.cfg` (or as an argument to `runscm`):

```
SCM_phydata=/path/to/input/file
```

Additional information about the input file (including its format, which defaults to netcdf), can be provided in the `&phydata` nameslist of the `scm_settings.nml` file. For more details about the possible entries in the namelist, see the `scm_settings.nml` documentation.

Once a file has been specified, a field can be read into any scalar (i.e. single-level variable) anywhere in the RPN Physics package by use-associating the phydata module and calling `pd_get` (the "pd" prefix is for "phydata"). The interface for this function is:

```
  function pd_get(F_name,stat) result(F_value)
    character(len=*), intent(in) :: F_name             !Field name in the input file
```

```
    integer, target, intent(out), optional :: stat      !Completion status of the function (GD_OK or GD_ERROR)
    TYPE :: F_value                                     !Value of the field of type TYPE (currently "real")
```

Examples

To read in the value of a variable named "Tg" (ground temperature), stored in a netcdf file whose name has been added to the `configexp.cfg` configuration file as the `SCM_phydata` entry, the following additions to any physics routine are all that is required to obtain the value at the current time level. In this case, the hypothetical subroutine "foo.ftn90" and variable BAR (a vector of length NI, but remembering that NI=1 for the SCM) is used as an example:

```
  subroutine foo(...)
    use phydata_mod
    ...
    BAR(1) = pd_get('Tg')
    ...
```

For simplicity in this example, no error checking is performed in this call. The `stat` optional argument is intended to allow for error checking. The optional argument should be provided with the `stat` keyword and a local variable (for example `icode`. Such a check might look like:

```
    BAR(1) = pd_get('Tg',stat=icode)
    if (icode /= PD_OK) then ...
```

With this call to `pd_get()` the value of BAR(1) is set to that obtained from the input file, and any error is handled as appropriate in the calling program.

Experiment Management

A minimal set of development tools will assist you with the the creation and maintenance of experiment (working) directories. Each of these tools accepts a `-h` flag that displays a list of available options. Experiments should be created with `scmdev.dot` as described in the setup instructions, a utility that creates space for developing, building and running the SCM in locations controlled by the environment. Only a single experiment can be active in a shell at a given time. To activate a different experiment, "dot" (source) the `.ssmuse_scm` setup in the experiment directory. Because the experiment structure is likely spread across several file systems, with directory names that may include hashes to

make them unique, a thorough cleanup of the experiment requires the use of `devrm`. By default, `devrm` prompts before removing any directories; however, this behaviour can be overridden with the `-f` (force removal) flag.

Examples

Creation of a new experiment (called MY_EXP in this example) is accomplished by "dotting" (sourcing) `scmdev`.

```
scmdev MY_EXP -case caribbean -v -cd -f
```

- [`MY_EXP`] gives the experiment the name MY_EXP. This required positional argument can also appear following a double-dash (--) on the command line.
- [`-case caribbean`] loads the "caribbean" benchmark case rather than the default GABLS-3 benchmark (use `scmcase -list` to retrieve a list of available benchmarks).
- [`-v`] executes the command with mild verbosity to keep you entertained.
- [`-cd`] leaves you in the newly created experiment directory. If this flag is not provided, then the experiment directory name is emitted for reference.
- [`-f`] forces the creation of the experiment, even if an experiment using the same SCM version with the same name already exists.

Cleanup of the experiment is difficult complete manually, so the `devrm` tool is provided for convenience.

```
devrm
```

The utility will prompt you for the removal of individual components of the currently active experiment. You may choose to keep some parts of the experiment while removing others.

Knowledge Base

This section contains a set of tips and tricks that will help the user to diagnose problems with an SCM setup.

- **The SCM runs slowly on my machine**: Depending on the output interval and number of variables requested in your `scm_settings.nml` file, the SCM may be performing over a hundred file open and close operations for each time step. For this reason, it is important that the `RUNSCM` directory be linked to a local storage device. This link is created when you run `linkit` based on the `$storage_model` environment variable. You can either change the value of `$storage_model` and rerun `linkit` or you can manually relink the `RUNSCM` directory to point to a local storage device to fix the SCM slowdown. By default, the environment variable `$TMPDIR` points to

temporary storage space on a local disk (remember, however, that this space will be removed when you log out of the shell in which you are working) and can be used to create some fast scratch space for the SCM. After establishing your working directory, the following steps allow you relink your SCM run directory for better performance.

```
mkdir -p $TMPDIR/$(basename $(pwd))/RUNSCM; ln -sf $TMPDIR/$(basename $(pwd))/RUNSCM .
```

- **`prepscm` tells me that it, "cannot find required field WW" in my analysis**: Analyses from CMC do not contain pressure-coordinate vertical motion fields, which are required as a prescribed input for the SCM. To create a realistic vertical motion field, GEM should be run and "WW" output directly. For SCM tests that do not require vertical motion, the following steps can be followed to generate a null (zero) vertical motion field (`ANALYSIS` should be replaced with the name of the analysis file that you are using).

```
my_analysis=ANALYSIS
editfst+ -s $my_analysis -d .tmp.fst <<EOF
 desire(-1,'TT')
 zap(-1,'WW')
EOF
setfld .tmp.fst $my_analysis WW=0.; rm -f .tmp.fst
```

- **Why don't my SCM profiles look exactly like the GEM profiles for my chosen gridpoint?**: Ideally, the SCM will represent the general character of a GEM integration when the SCM is initialized and forced with GEM model data; however, there are a number of reasons that even the best-designed SCM integration will not reproduce the details of the profiles and time series generated by GEM. The longer the integration, and the more sensitive the profile to small changes, the larger the difference in evolution will be between the two models.
  - Initial conditions taken from step-0 of a driving integration have likely been numerically truncated when the GEM output file was written. This means that the initializing profile of the SCM will not match perfectly with the step-0 GEM profile even if the "nearest" interpolation option is chosen for `prepscm` to prepare the input files.
  - If dynamics tendencies are computed by `prepscm` rather than being output from GEM, the accuracy of the forcings calculations will affect the SCM results. The forcing calculations are particularly problematic over steep terrain, where rectification to pressure levels for the computations may introduce noticeable errors.
  - If the dynamics tendencies ("*TND" or "*DYN") are not provided at every timestep, then the linear time interpolation of the forcing fields will necessarily create differences between the GEM and SCM solutions.
  - In GEM, geopotential heights are computed from a basic state ϕ^* and a perturbation ϕ'. These variables exist in the GEM model state vector, but not in the SCM where only the "physical world" state is advanced. As a result, the SCM uses a form of the hypsometric equation to compute geopotential heights, leading to small differences in this variable on the dynamics bus (input to the physics) at each timestep.
  - The precision of the SCM profile location will affect the interpolation of the surface analysis inputs performed by the physics. The order of this interpolation can be controlled by the physics input table, which is identified using the `-SCM_phy_intable` argument to `runscm`.

    - There are three steps in GEM that modify the "physical world" variables that comprise the state vector of the SCM: dynamics (advection and pressure solver), physics and diffusion. The steps occur in the order listed. The full "dynamics" tendency values ("*TND") include all changes made to the physical world over the course of GEM step outside of the physics (e.g. by the advection, pressure solver and diffusion); however, it is impossible to split these tendencies up into parts that occur before the physics and those than occur after the physics. Since the tendencies are applied at the beginning of the timestep as part of the SCM dynamics, it is as if the diffusion is also being run before the physics. In general, the influence of this inconsistency between GEM and the SCM is small since diffusion is generally small in GEM; however, for very sensitive profiles the impact may be noticeable.
    - Large differences between GEM and SCM results can occur when the "dynamics" tendency values ("*TND") are extracted from a GEM run with the digital filter applied. At the digital filter reset time step, all tendencies for the last half of the filter period appear in the "*TND" values. This leads to a shock in the tendencies for the SCM that is an artifact of the forcing fields. To avoid this problem: (1) GEM could be run without the digital filter, (2) the filter reset timestep could be ignored with a judicious use of `fcst_nesdt_s`, or (3) the physics-produced forcings could be used ("*ADV").
    - The use of smoothing operators on physics inputs (e.g. `cond_infilter`) or of the tendencies (e.g. `sgo_tdfilter`) may lead to significant differences when GEM integrations are compared with SCM results. This is because horizontal operations are not possible in the column model, so these operations are implemented as copies rather than horizontal smoothing.
- **When I try to compile my physics code using the phydata package, I get an error about an "Corrupt or Old Module file /usr/include/netcdf.mod"**: The base netcdf installation was built with an older version of the PGI compiler whose module files are not compatible with the current version. Run the following command to obtain a more up-to-date version of the netcdf build (this can be added to your interactive login profile as well):

```
. s.ssmuse.dot ENV/netcdf/3.6.0
```

- **When I try to build the SCM with `make scm`, I get a bunch of undefined references to things starting with "netcdf_nf90_"**: See the previous entry regarding the netcdf module file. The problem here is that the old library does not contain these entry points. The solution (i.e. `. s.ssmuse.dot ENV/netcdf/3.6.0`) is exactly the same as above.
- **How do I know which version of SCM I was using for my experiment?**: If you opened your experiment with `ouv_exp_scm`, then a file has been created in your experiment directory called `.ssmuse_scm`. To re-acquire the version of the SCM that you opened this experiment with, simply "dot" (source) that file and the original SCM environment will be restored.
- **How do I get additional debugging information?**: Since the SCM is a relatively simple modelling framework, debugging information can be useful for identifying the sources of potential problems. Using the `-SCM_debug` and (optionally) `-SCM_supp` flags to `runscm` on Linux platforms will activate the Valgrind debugger (other debugger names can be specified as an argument to `-SCM_debug`) with default suppressions supplied by the RPN physics.
- **What happens when I turn `dyn_forcings` off?**: The `dyn_forcings` setting in the `&scm_cfgs` namelist of the SCM configuration file can be used to turn off all terms in the SCM equations

except those arising from the physics. Time-varying prescribed values (vertical motion and surface pressure) continue to evolve as in a dynamically forced simulation.
- **How do I hold my forcings constant?**: To hold forcings constant at their initial values, set `Fcst_nesdt_S` to a negative value in the `&step` namelist of the SCM configuration file. In addition to holding all dynamics forcings constant, this will prevent the prescribed fields (vertical motion and surface pressure) from varying in time.
- **My `runscm` command returns an error message that looks like "`task_setup.ksh: not found`"**: The `task_setup` utility used by the SCM is provided by Maestro and therefore requires the sequencer to be loaded into the user environment. Consult the Maestro documentation for instructions on how to load the sequencer package.

References

1. The "*TND" fields are generated by GEM v4.4.0 and above only.